

Lab6 实验报告

姓名：朱文凯

学号：23307110192

1. 实验目标

本次 Lab6 的目标是在已有五级流水线 CPU 和 Lab5 特权级切换基础上，补充 RISC-V 机器模式下的异常与中断处理能力，并通过 `ready-to-run/lab6/lab6-test.bin` 测试。

本次实现覆盖的内容包括：

- 异常：ecall、指令地址不对齐、load/store 地址不对齐、非法指令。
- 中断：机器软件中断、机器时钟中断、机器外部中断。
- CSR 状态更新：mepc、mcause、mtval、mstatus、mie、mip 等相关路径。
- 流水线控制：在 WB 提交边界统一进入 trap，并清除年轻指令，避免异常后错误路径继续提交。

本次没有实现 Bonus 的缺页异常。

2. 总体设计思路

我采用的核心思路是：异常和中断都不在前级直接修改 CSR，而是先形成一组 trap 信息，并随流水线传递到 WB 阶段，在提交边界统一更新 CSR 和重定向 PC。

这样做主要有两个原因。第一，WB 阶段已经是当前 CPU 的架构提交点，寄存器写回、CSR 写回和 Difftest 提交都集中在这里，适合处理精确异常。第二，如果在 EX 或 MEM 阶段直接跳转，前面已经进入流水线的指令可能仍然产生写回或访存副作用，容易破坏异常的精确性。

因此，本次实现中每条可能触发 trap 的指令都会携带：

- exception_valid
- exception_cause
- exception_tval

到 WB 后再根据优先级决定是普通提交、异常 trap、ecall trap、mret 返回，还是异步中断 trap。

3. 异常实现

3.1 非法指令

在 `decode_out_t` 中增加 `is_illegal` 字段，并在译码阶段识别明显非法的 `system` 指令和空指令。非法指令进入流水线后被转换为 `CAUSE_ILLEGAL_INST`，最终在 WB 阶段提交异常。

同时，`sfence.vma` 在本设计中作为 `no-op` 放行。原因是 Lab5 内核会使用该指令，而当前 MMU 模型不需要额外执行 TLB 刷新动作。

3.2 指令地址不对齐

跳转和分支目标在 EX 阶段计算。若实际发生跳转，并且目标地址不是 4 字节对齐，则产生 `instruction address misaligned` 异常：

- `mcause = 0`
- `mtval =` 错误的跳转目标地址
- `mepc =` 触发跳转的指令 PC

若分支不成立，则不会检查目标地址不对齐，因为该目标不会真正被取指。

3.3 数据地址不对齐

`load/store` 地址同样在 EX 阶段由 ALU 算出。根据访存宽度判断对齐要求：

- `byte` 访问不要求额外对齐
- `half word` 要求 2 字节对齐
- `word` 要求 4 字节对齐
- `double word` 要求 8 字节对齐

若发现地址不对齐，会产生 `load/store address misaligned` 异常，并且在 MEM 阶段屏蔽 `dreq.valid`，避免错误 `store` 真的写入内存。

4. 中断实现

中断 `pending` 来源于顶层输入：

- `swint` 对应 MSIP
- `trint` 对应 MTIP
- `exint` 对应 MEIP

core 内部将硬件 `pending` 与 CSR 中的软件 `pending` 合并后，与 `mie` 做按位与，得到当前可响应的中断集合。为了不影响 Lab4/Lab5 的 `Difftest` CSR 比对，硬件 `pending` 只参与 core 内部仲裁，不强行写入 `Difftest` 可见的 `mip` CSR 输出。

中断使能条件为：

- 当前不在 M Mode；或
- 当前在 M Mode 且 `mstatus.MIE = 1`

当 WB 阶段提交一条普通指令且中断条件满足时，当前指令先正常提交，然后进入中断 `trap`：

- `mepc = pc + 4`
- `mcause[63] = 1`
- `mcause[62:0]` 为对应中断号

- `mstatus.MPIE = mstatus.MIE`
- `mstatus.MIE = 0`
- `mstatus.MPP = 当前特权级`
- `priv_mode = M`

中断优先级按外部中断、时钟中断、软件中断的顺序选择。

5. 流水线控制

本次实现中，异常、`ecall`、`mret` 和中断都通过统一的 `system event` 路径处理。当前端或中间级发现系统事件后，会清空年轻指令，防止它们在 `trap` 前继续提交。

一个关键细节是：系统事件不能在还没有进入后级前就被 `flush` 掉。因此 `ID/EX` 的清空条件需要和 `EX/MEM` 的推进条件配合，只有在事件能继续向后流动时才清除当前位置。否则会出现异常指令丢失、PC 停住或 `trap` 不触发的问题。

另一个关键点是 `WB` 阶段的等待。若 `trap/mret` 准备提交时前面仍有 `fetch` 或 `memory` 等待，先保持 `WB` 中的系统事件，等总线状态稳定后再更新 `CSR` 和 `PC`。为避免组合环，使用上一拍的 `fetch_wait`、`mem_wait` 作为 `redirect ready` 判断。

6. 调试中遇到的问题

最主要的问题有三个：

1. 错位 `load/store` 被识别为异常后，如果仍然让 `mem_access_mem` 为真，就会因为 `dreq.valid` 被屏蔽而永远等不到 `dresp.data_ok`。解决方法是异常访存不再参与 `mem_access_mem`。
2. 中断 `pending` 如果直接写进 `CSR mip` 输出，会破坏 `Lab4/Lab5` 中 `DiffTest` 对 `CSR` 的比较。最终采用的做法是：`CSR` 可见状态保持原有行为，硬件 `pending` 只在 `core` 内部参与中断判断。
3. `Lab5` 中的 `PMP CSR` 和 `sfence.vma` 需要按现有项目边界处理。`PMP CSR` 对 `NEMU` 来说可能触发非法路径，因此在 `DiffTest` 提交中跳过；`sfence.vma` 对当前简单 `MMU` 模型作为 `no-op`。

7. 验证结果

本次主要运行了以下测试：

- `make sim`：构建通过。
- `make test-lab4`：通过，关键输出为 `HIT GOOD TRAP`。
- `timeout 25s make test-lab5`：通过，关键输出为 `Return from init! Test passed`。
- `TEST=sys ./build/emu --no-diff -i ./ready-to-run/lab6/lab6-test.bin -C 8000000 --force-dump-result`：通过，关键输出包括：
 - `Test ecall_u [OK]`
 - `Test instr_misalign [OK]`
 - `Test load_misalign [OK]`
 - `Test store_misalign [OK]`

- Test timer_intr [OK]
- Test software_intr [OK]
- Test m_trap [OK]
- Privileged test finished.
- Exit with code = 0

Lab6 测试在打印完成后仍可能继续运行到 cycle limit，这是测试框架行为；判断通过主要看 Privileged test finished. 和 Exit with code = 0。

8. 总结

本次实验完成了异常和中断的基本支持。实现过程中我尽量把 trap 处理集中在 WB 提交边界，避免前级直接修改架构状态带来的不精确问题。最终 Lab6 功能测试通过，并且 Lab4、Lab5 回归没有被破坏。